Guía de defensa · Parcial 01 (uso interno del equipo)

La defensa vale **40 %** de la nota: 10 % es la ejecución en vivo, 7 % la defensa del Data Mart, 7 % la defensa de la BDOO y 6 % el dominio individual. El profesor puede preguntarle **a cualquiera** por **cualquier** parte, así que todos deben leer las secciones 1 a 5 de esta guía.

Reparto de la exposición (10 min)

Láminas	Quién	Tiempo	Mensaje que debe quedar
1–3 Portada, reto, arquitectura	Mathias	2:30	Tres responsabilidades separadas: el Data Mart tiene la verdad, la BDOO la trazabilidad y el agente conecta ambos.
4–5 Estrella y decisiones	Mikel	2:30	El grano en una oración, la aditividad, las claves sustitutas y los miembros −1/−2.
6–7 SCD2 y controles	Leonardo	2:00	Con Tipo 1 el agente respondería «Frecuente», que es falso. La conciliación cuadra al centavo.
8–9 BDOO	Federico	2:00	Cada concepto de la clase P06 está implementado y demostrado.
10, 12, 13 Agente, pruebas y cierre	Fernando	1:30	El agente nunca inventa: tiene cuatro capas de seguridad. Pruebas 35/35 contra valores calculados por separado, más una revisión independiente.
11 Demostración	Mathias opera; cada uno explica su parte	2:00	Seguir el guion de la lámina 11.

Antes de entrar (checklist)

1. Correr `python scripts/setup_datamart.py`: debe terminar en **TODO OK**.
2. Correr `python scripts/inicializar_bdoo.py`.
3. Abrir `streamlit run app/streamlit_app.py` y hacer una pregunta de prueba.
4. Si hay internet, dejar el modo **auto**. Si falla el LLM o no hay red, usar **reglas**, que da el mismo resultado en las preguntas del guion.
5. Tener la terminal lista con `python -m agente.cli -p "…"` como plan B. Ojo: cierren Streamlit antes, porque ZODB admite un solo proceso.
6. Subir la última versión del .zip a Canvas **antes del viernes a las 5:00 p. m.**

---

1. Preguntas probables · Data Mart (Mikel y Leonardo)

¿Cuál es el grano? Una fila por cada línea de producto de un ticket de venta completado.

¿Por qué ese grano y no el ticket? Es el más atómico. Con él se puede responder por producto y por categoría, y agregar hacia arriba. Con grano ticket no sabríamos qué productos se vendieron.

¿Cómo evitan duplicar ingresos? De cuatro formas: - La PK `(numero_ticket, linea_ticket)`. - El ETL deduplica la fuente (había 384 líneas repetidas). - Los tickets se cuentan con `COUNT(DISTINCT numero_ticket)`, nunca con `COUNT(*)`. - La conciliación C1–C7 lo comprueba.

Métricas aditivas, semiaditivas y no aditivas: - **Aditivas:** cantidad, monto bruto, descuento, ingreso neto, costo y margen. - **No aditivas:** margen %, ticket promedio, % de descuento, clientes activos y precio unitario. - **Semiaditivas:** no hay en un hecho transaccional. El ejemplo sería el inventario: se suma por producto, pero no a lo largo del tiempo.

¿Por qué no se promedia el margen %? Porque un canal pequeño pesaría igual que uno grande. Se calcula como `SUM(margen) / SUM(ingreso)`. Por eso la comparación entre canales usa `GROUPING SETS`, que recalcula el total del periodo.

¿Para qué sirven las claves sustitutas? - Independizan el DW de los códigos de la fuente. - Permiten varias versiones de un cliente (SCD2). - Permiten los miembros −1/−2. - Hacen los joins más rápidos.

`fecha_sk` es AAAAMMDD por la convención de Kimball.

¿Qué pasa con un producto que no existe en el catálogo? Va a `producto_sk = −2` («Sin correspondencia»). No se descarta, porque perderíamos ingreso: son 202 líneas, 0,21 % del ingreso. La regla RV09 advierte al usuario cuando aparece en un resultado.

¿Y los clientes sin identificar? Van a −1 («No informado»): 792 tickets de consumidor final.

¿SCD 1 o 2? (pregunta obligatoria) Tipo 2. Si se sobrescribiera, las compras de 2025 de los corporativos reclasificados contarían como «Frecuente». El agente respondería que Frecuente cayó −31,5 % y que Corporativo creció +5,5 %, cuando en realidad Corporativo cayó −20,6 %. Mostrar la consulta Q4-bis del .sql.

¿Cómo se carga el SCD2? Desde la bitácora del CRM (`oltp.cliente_segmento_historial`): una versión por tramo, con `fecha_inicio`, `fecha_fin` y `es_actual`. Cada venta se une con la versión vigente en su fecha (`BETWEEN fecha_inicio AND fecha_fin`). El control I9 verifica que ningún hecho quede fuera de vigencia, e I7/I8 que no haya solapes.

¿Qué diferencia hay entre oltp y dm? oltp es operacional: normalizado, con errores y sobrescribe. dm es analítico: estrella limpia, histórica y no volátil. El agente no puede leer oltp ni control (REVOKE + RV07).

¿Cómo concilian? Con la vista `control.v_conciliacion_estado`, que compara líneas, tickets, unidades, ingreso, costo, el cuadre de líneas crudas y la diferencia mes a mes. Además, `setup_datamart.py` recalcula los totales con pandas directo de los CSV.

Kimball o Inmon? Kimball (bottom-up): un Data Mart de ventas con dimensiones conformadas, reutilizables para un futuro Data Mart de inventario.

2. Preguntas probables · BDOO (Federico; todos deben poder responder lo básico)

¿Por qué ZODB? Es una BDOO real: persiste objetos Python sin mapeo a tablas, asigna IDO y guarda las referencias entre objetos. Tiene herencia y polimorfismo transparentes, Blobs (LOB), transacciones ACID e historial de versiones.

¿Por qué no PostgreSQL objeto-relacional? Habría mezclado la BDOO con el Data Mart, y la rúbrica castiga confundir esas responsabilidades.

¿La BDOO duplica las dimensiones? No. Guarda objetos del agente: sesiones, preguntas, intenciones, consultas, resultados, respuestas, reglas y fuentes. Las cifras viven en el Data Mart. Lo único que se guarda del dato es el resultado de cada consulta como evidencia.

¿Qué es el IDO? El `_p_oid` que asigna ZODB. Es técnico, no tiene significado de negocio y no cambia. Se ve en la pestaña BDOO de la app.

Ejemplos concretos de herencia y polimorfismo: - `Usuario` es abstracta; `Gerente` y `Analista` implementan `nivel_detalle()` y `muestra_sql()` distinto (el analista ve el SQL). - `ConsultaAnalitica` tiene 5 subclases: el agente les envía `aplica_a()` a todas y `construir_sql()` a la elegida. - `ReglaValidacion` tiene 11 subclases con `validar()` propio.

Es el mismo caso de Puerta/Ventana con «Cerrar» de la clase.

Encapsulamiento: los atributos son privados (`_estado`), solo hay propiedades de lectura y el estado cambia mediante métodos que validan. Ejemplo: `Pregunta.cambiar_estado()` no permite pasar de RECIBIDA a RESPONDIDA sin INTERPRETADA.

Composición vs. agregación: - **Composición:** Sesión ◆ Preguntas; Pregunta ◆ Intención y Respuesta; Respuesta ◆ Resultado. La parte no existe sin el todo. - **Agregación:** el agente ◇ el catálogo y las reglas; la Respuesta → FuenteDato es compartida.

Apuntadores suaves: ZODB guarda el OID del objeto referenciado y lo carga cuando se accede (*lazy*). Por eso dos respuestas apuntan al *mismo* objeto FuenteDato (`is` = True).

LOB y TDA: - **LOB:** `ResultadoConsulta` guarda el CSV completo en un Blob. - **TDA:** `DocumentoIdentidad.formato()` valida «V-…» igual que el TDA DATOS del ejemplo de Oracle. `Periodo` y `Filtro` también son TDA inmutables con validación.

Versionamiento: `ConsultaAnalitica.nueva_version(nota)` guarda el historial de forma explícita, y FileStorage conserva cada revisión (`db.history(oid)`). Está en el punto 6 de `docs/evidencias/evidencia_bdoo.md`.

¿Qué pasa si falla algo a mitad de una pregunta? La transacción hace `abort()` y no queda nada a medias. Ante un conflicto de concurrencia (`ConflictError`) se reintenta.

¿Qué guarda la memoria de la sesión? La última intención ejecutable. Por eso «¿y en 2025?» funciona.

3. Preguntas probables · Agente (Fernando y Mathias)

¿El LLM escribe SQL? No. Solo devuelve un JSON con códigos del catálogo. El SQL sale de plantillas con listas blancas y los valores viajan como parámetros.

¿Y si el LLM se equivoca o inventa? Su JSON se valida (`construir_intencion` + RV02–RV06). Si menciona una cifra que no está en el resultado, la narrativa se descarta (RV11). Si falla, se usan las reglas.

Las cuatro capas de seguridad: 1. RV01 sobre el texto. 2. El LLM sin SQL. 3. RV07 sobre el SQL final. 4. El rol `agente_lector`: solo SELECT, READ ONLY y timeout de 5 s.

Aunque alguien lograra saltar una capa, la base de datos sigue rechazando escrituras.

¿Cómo manejan una pregunta ambigua? Con una ACLARACION que propone criterios (ingreso, margen, unidades o ticket). No elige por el usuario.

¿Qué hacen con una pregunta sin periodo? Usan todo el rango y lo advierten de forma explícita.

¿Y si no hay registros? Responden SIN_DATOS: «el resultado está vacío; no se reporta como cero».

¿Cómo eligen los «últimos tres meses»? Sobre la fecha máxima del Data Mart (30-09-2026), no sobre la fecha de hoy. Son jul–sep 2026.

¿Cómo saben que las respuestas son correctas? La matriz de 35 casos compara contra valores calculados con pandas directamente sobre los CSV (`scripts/ejecutar_pruebas.py`). Además, un revisor independiente probó más de 120 preguntas. Encontró, por ejemplo, que el ranking de clientes agrupaba por nombre (había empresas distintas con la misma razón social). Se corrigió agrupando por `cliente_id`, y cada defecto quedó como caso de regresión (P25–P34).

¿Por qué el agente dice que «método de pago» no existe, si está en la fuente? Porque lo dejamos fuera del Data Mart a propósito: ninguna pregunta lo pide. El agente responde con lo que hay en el Data Mart y no inventa. Agregarlo sería una dimensión pequeña (junk dimension) sin cambiar el grano.

¿Por qué «¿Qué productos debemos eliminar del catálogo?» no se bloquea? RV01 distingue una **orden** («Borra las ventas…») de una **pregunta** de negocio que usa el mismo verbo. Las instrucciones SQL (DROP, DELETE FROM…) y las credenciales se bloquean siempre.

¿Qué pasa si comparo 2026 con 2025? 2026 solo tiene datos hasta septiembre, así que el agente compara ene–sep contra ene–sep y lo dice. Comparar un año parcial contra uno completo daría una caída falsa.

4. «Modifiquen algo en vivo» (dominio individual)

- **Agregar una métrica** (por ejemplo, precio promedio):
- En `agente/catalogo.py`, agregar a `METRICAS`: `Metrica("precio_promedio", "Precio promedio", "ROUND(SUM(v.monto_bruto)/NULLIF(SUM(v.cantidad),0),2)", "no aditiva", "usd", "SUM(monto_bruto)/SUM(cantidad)", ("monto_bruto","cantidad"), "analizar_ventas")`.
- En `agente/interpretes/reglas_nl.py`, agregar a `METRICAS_PATRONES`: `(r"precio promedio", "precio_promedio")` **antes** de la línea de `ingreso_neto`.
- Reiniciar la app.
- **Agregar una dimensión existente en la tabla** (por ejemplo, semana): agregarla a `DIMENSIONES` (`"f", "semana_iso"`) y un patrón en `DIMENSIONES_PATRONES`.
- **Desactivar una regla:** en la consola de Python, `agente.regla("RV09").activar(False)` dentro de una transacción. Se persiste porque es un objeto de la BDOO.
- **Ver la historia de un objeto:** en la pestaña BDOO de la app, pegar el IDO de una consulta.
- **Explicar un SQL:** abrir «Evidencia y trazabilidad» y leer FROM, JOIN, WHERE con parámetros, GROUP BY y ORDER BY con LIMIT.

5. Cifras para tener a mano

- Fuente: 30.970 tickets y 99.890 líneas. Data Mart: 97.816 líneas y 30.462 tickets. Ingreso neto: USD 3.494.922,06.
- Rechazos: 495 tickets anulados, 384 líneas duplicadas y 108 con cantidad inválida.
- Clientes: 2.360 clientes y 2.736 versiones; 376 con cambio de segmento.
- Canal con mayor venta: Mayorista B2B (45 %). Sucursal con mayor ticket promedio: Chacao. Segmento que más cayó: Corporativo (−20,6 %).

- 11 reglas, 5 consultas del catálogo, 10 clases base y 18 subclases. Pruebas: 35/35 en la matriz y 35/35 unitarias.
- Los 10 primeros clientes suman ≈4,2 % del ingreso: la dependencia de pocos clientes es baja.